

Regresión Polinomial en Python

Comparación de grados, generalización y regularización

M en C. Jeoshua Alejandro Licea Garcia

Resumen

Este mini proyecto muestra cómo implementar regresiones polinomiales utilizando diferentes grados y cómo comparar su desempeño mediante el coeficiente de determinación R^2 . El objetivo es identificar modelos que presentan *underfitting* (subajuste), modelos con una buena capacidad de generalización y modelos que presentan *overfitting* (sobreajuste). La implementación utiliza arreglos de NumPy, vectorización y herramientas de `scikit-learn`.

Índice

1. Introducción	2
2. Generación de los datos	2
2.1. Explicación	2
3. Visualización del conjunto de datos	3
4. Predicciones para diferentes grados	3
4.1. ¿Qué ocurre en esta función?	4
5. Comparación mediante R^2	4
6. Análisis de underfitting y overfitting	5
6.1. Underfitting	6
6.2. Buena generalización	6
6.3. Overfitting	6
7. Regularización con Lasso	7
8. Conclusiones	7

1. Introducción

La regresión polinomial es una extensión de la regresión lineal que permite modelar relaciones no lineales entre una variable independiente x y una variable dependiente y . Aunque el modelo resultante es no lineal respecto de x , sigue siendo lineal respecto de sus parámetros:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n.$$

En este proyecto se generan datos sintéticos a partir de una función seno, añadiendo una tendencia lineal y ruido aleatorio. Posteriormente, los datos se dividen en conjuntos de entrenamiento y prueba. Se ajustan modelos polinomiales de distintos grados y se estudia cómo cambia su desempeño.

El análisis permite observar el compromiso entre complejidad y capacidad de generalización: un modelo demasiado sencillo puede ser incapaz de representar la estructura de los datos, mientras que uno demasiado complejo puede adaptarse excesivamente al conjunto de entrenamiento.

2. Generación de los datos

Primero se importan las bibliotecas necesarias y se fija una semilla para que los resultados sean reproducibles.

```
1 %matplotlib inline
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6
7 np.random.seed(0)
8
9 n = 15
10 x = np.linspace(0, 10, n) + np.random.randn(n) / 5
11 y = np.sin(x) + x / 6 + np.random.randn(n) / 10
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     x, y, random_state=0
15 )
```

2.1. Explicación

- `np.random.seed(0)` fija la semilla del generador aleatorio.
- `np.linspace(0,10,n)` genera 15 puntos distribuidos entre 0 y 10.
- Se agrega ruido mediante `np.random.randn(n)/5`.
- La variable objetivo se construye como

$$y = \sin(x) + \frac{x}{6} + \text{ruido}.$$

- `train_test_split` separa los datos en entrenamiento y prueba.

La semilla es importante porque permite obtener la misma partición y los mismos datos aleatorios al volver a ejecutar el programa (reproducibilidad).

3. Visualización del conjunto de datos

La función `intro()` muestra gráficamente los puntos utilizados para entrenar y evaluar los modelos.

```
1 def intro():
2     plt.figure()
3     plt.scatter(X_train, y_train, label='training data')
4     plt.scatter(X_test, y_test, label='test data')
5     plt.legend(loc=4)
6     plt.show()
7
8 intro()
```

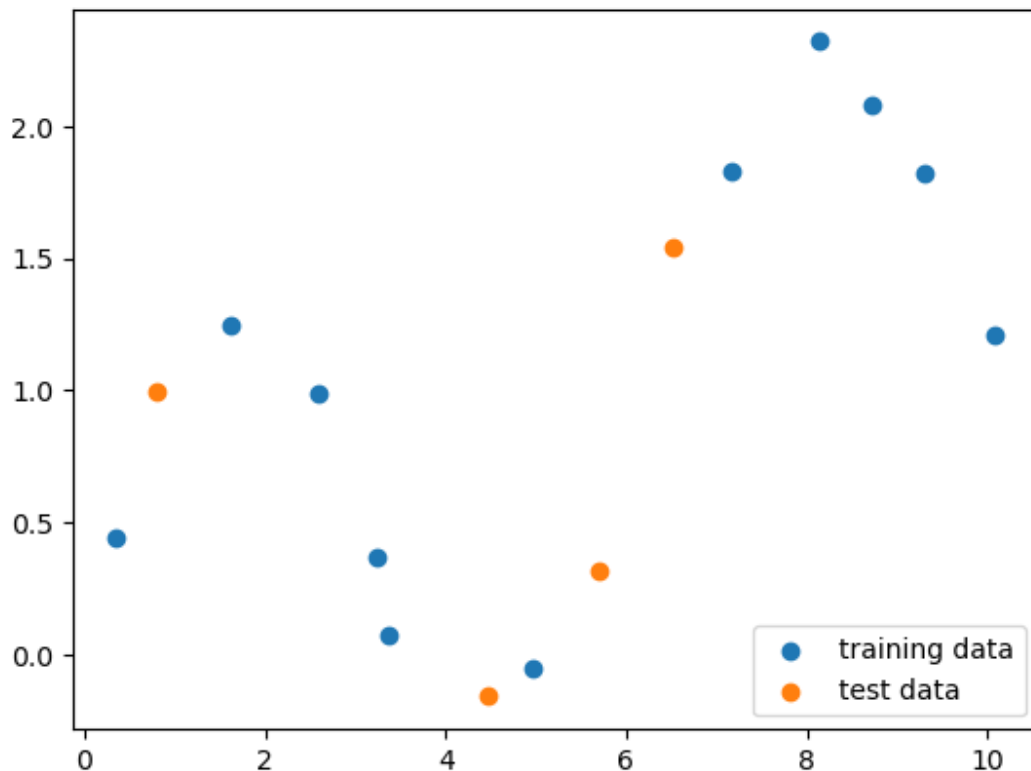


Figura 1: Separación de los datos en entrenamiento y prueba.

La figura 1 permite comprobar visualmente que existen dos subconjuntos: los datos de entrenamiento, utilizados para ajustar los parámetros del modelo, y los datos de prueba, utilizados para estimar cómo funciona el modelo con observaciones que no utilizó durante el entrenamiento.

4. Predicciones para diferentes grados

La función `answer_one()` calcula predicciones para polinomios de grado 1, 3, 6 y 9.

```
1 def answer_one():
2     import numpy as np
3     from sklearn.linear_model import LinearRegression
4     from sklearn.preprocessing import PolynomialFeatures
5
6     x_p = np.linspace(0, 10, 100).reshape(-1, 1)
7
8     X_train, X_test, y_train, y_test = train_test_split(
```

```

9         x, y, random_state=0)
10
11     X_train = X_train.reshape(-1, 1)
12     X_test = X_test.reshape(-1, 1)
13
14     degree_predictions = np.zeros((4, 100))
15     j = 0
16
17     for i in [1, 3, 6, 9]:
18         poly = PolynomialFeatures(degree=i)
19         X_train_poly = poly.fit_transform(X_train)
20         x_p_poly = poly.transform(x_p)
21
22         polireg = LinearRegression().fit(
23             X_train_poly,
24             y_train
25         )
26
27         degree_predictions[j, :] = polireg.predict(x_p_poly)
28         j += 1
29
30     return degree_predictions
31
32 answer_one()

```

4.1. ¿Qué ocurre en esta función?

La instrucción

```
1 PolynomialFeatures(degree=i)
```

transforma cada entrada x en un conjunto de características de la forma

$$[1, x, x^2, \dots, x^i].$$

Por ejemplo, para un polinomio de grado 3, la representación de una entrada x es

$$[1, x, x^2, x^3].$$

El arreglo

$$\text{degree_predictions} \in R^{4 \times 100}$$

almacena las 100 predicciones correspondientes a cada uno de los cuatro grados analizados. Esto es un ejemplo de uso de arreglos y vectorización: en lugar de guardar cada predicción de manera individual, se organizan todas las predicciones en una matriz.

5. Comparación mediante R^2

El siguiente bloque evalúa modelos con grados de 0 a 9.

```

1 def answer_two():
2     from sklearn.linear_model import LinearRegression
3     from sklearn.preprocessing import PolynomialFeatures
4
5     X_train, X_test, y_train, y_test = train_test_split(
6         x, y, random_state=0
7     )
8
9     X_train = X_train.reshape(-1, 1)
10    X_test = X_test.reshape(-1, 1)

```

```

11
12     r2_train = np.zeros(10)
13     r2_test = np.zeros(10)
14
15     for j, i in enumerate(range(10)):
16         poly = PolynomialFeatures(degree=i)
17         X_train_poly = poly.fit_transform(X_train)
18         X_test_poly = poly.transform(X_test)
19
20         polireg = LinearRegression().fit(
21             X_train_poly,
22             y_train)
23
24         r2_train[j] = polireg.score(
25             X_train_poly,
26             y_train)
27
28         r2_test[j] = polireg.score(
29             X_test_poly,
30             y_test)
31
32     return r2_train, r2_test
33
34 answer_two()

```

El coeficiente de determinación se interpreta, de manera general, como una medida de qué tan bien el modelo explica la variabilidad observada. Su forma clásica es

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

En el código se calculan dos valores para cada grado:

- **r2_train**: desempeño sobre los datos usados para entrenar.
- **r2_test**: desempeño sobre datos que el modelo no utilizó para ajustar sus parámetros.

Esta comparación es especialmente útil para detectar sobreajuste.

6. Análisis de underfitting y overfitting

La función `answer_three()` genera la gráfica de R^2 para los grados de 0 a 9.

```

1 def answer_three():
2     r2_train, r2_test = answer_two()
3     grado = np.arange(10)
4
5     plt.figure(figsize=(8, 5))
6     plt.plot(grado, r2_train, marker='o',
7              label='Training R ')
8     plt.plot(grado, r2_test, marker='o',
9              label='Test R ')
10    plt.xlabel('Polynomial Degree')
11    plt.ylabel('R Score')
12    plt.title('Training vs Test R ')
13    plt.xticks(grado)
14    plt.legend()
15    plt.grid()
16    plt.show()
17
18    print(grado)
19

```

```

20     return print(
21         f'Underfitted Grado cero: {r2_test[0]:.3f}, '
22         f'Overfitted Grado nueve: {r2_test[9]:.3f}, '
23         f'Good generalization Grado sexto: {r2_test[6]:.3f}'
24     )
25
26 answer_three()

```

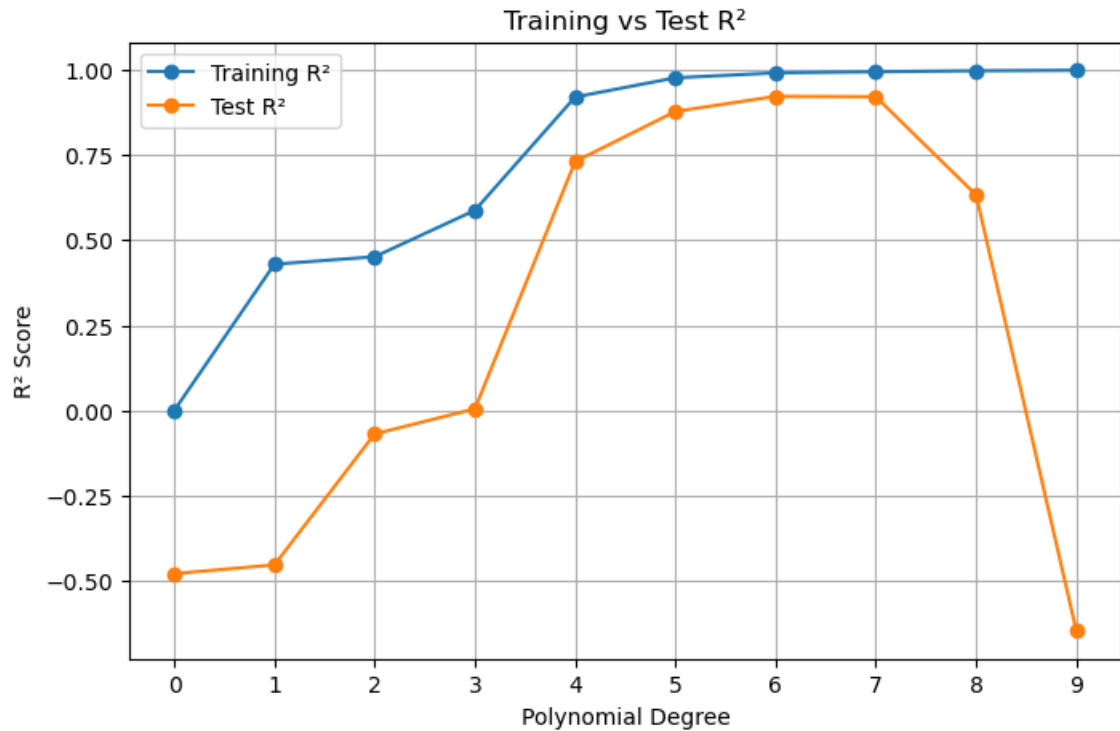


Figura 2: Comparación del desempeño de entrenamiento y prueba para distintos grados polinomiales.

La figura 2 muestra un comportamiento típico de selección de complejidad del modelo:

6.1. Underfitting

En grados muy bajos, el modelo tiene poca capacidad para representar la estructura de los datos. En particular, el grado 0 produce un R^2 de prueba negativo en esta ejecución. Esto indica que el modelo es demasiado simple para describir adecuadamente la relación entre x y y .

6.2. Buena generalización

Al aumentar el grado, el R^2 de prueba mejora. En los resultados mostrados, el grado 6 alcanza aproximadamente $R^2 = 0.92$ en prueba y cerca de $R^2 = 0.99$ en entrenamiento. La diferencia entre ambos valores es relativamente pequeña, por lo que este grado ofrece una buena combinación entre ajuste y generalización para este conjunto de datos.

6.3. Overfitting

En grados altos, el R^2 de entrenamiento continúa acercándose a 1, pero el desempeño sobre prueba puede comenzar a deteriorarse. Para el grado 9, la figura muestra aproximadamente

$$R^2_{\text{train}} \approx 1.00, \quad R^2_{\text{test}} \approx -0.65.$$

Esta diferencia es una señal clara de sobreajuste: el modelo aprende muy bien los datos de entrenamiento, pero no generaliza correctamente a los datos de prueba.

Nota: estos resultados dependen del conjunto de datos generado y de la partición aleatoria. Por ello, las conclusiones deben interpretarse como resultado de esta ejecución concreta y no como una propiedad universal de los grados 6 o 9.

7. Regularización con Lasso

Finalmente, `answer_four()` compara una regresión lineal aplicada a características polinomiales de grado 12 con una regresión Lasso.

```
1 def answer_four():
2     from sklearn.preprocessing import PolynomialFeatures
3     from sklearn.linear_model import Lasso, LinearRegression
4     from sklearn.metrics import r2_score
5
6     poly = PolynomialFeatures(degree=12)
7     X_poly = poly.fit_transform(x.reshape(-1, 1))
8
9     X_train, X_test, y_train, y_test = train_test_split(
10         X_poly, y, random_state=0
11     )
12
13     linreg = LinearRegression()
14     linreg.fit(X_train, y_train)
15     linreg_r2 = linreg.score(X_test, y_test)
16
17     lasso = Lasso(alpha=0.01, max_iter=10000, tol=0.1)
18     lasso.fit(X_train, y_train)
19     lasso_r2 = lasso.score(X_test, y_test)
20
21     print('(R2 Lineal, R2 Lasso)')
22     return (linreg_r2, lasso_r2)
23
24 answer_four()
```

La regresión Lasso agrega una penalización L_1 a los coeficientes del modelo. De manera simplificada, su función objetivo puede escribirse como

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - (\omega_i x_i + b))^2 + \alpha \sum_{j=1}^p |\beta_j| \right].$$

El parámetro `alpha=0.01` controla la intensidad de la penalización. Una consecuencia importante de Lasso es que puede reducir algunos coeficientes hacia cero, ayudando a controlar la complejidad de modelos polinomiales.

En este ejemplo se utiliza un polinomio de grado 12 deliberadamente complejo. La comparación entre `LinearRegression` y `Lasso` permite estudiar cómo una técnica de regularización puede ayudar a reducir los efectos del sobreajuste.

8. Conclusiones

El proyecto permite observar experimentalmente tres situaciones importantes:

1. **Modelo demasiado simple:** grados bajos pueden presentar *underfitting*, ya que no tienen suficiente capacidad para representar la relación existente en los datos.

2. **Modelo con buena generalización:** existe una región de grados intermedios donde el desempeño de entrenamiento y prueba es simultáneamente alto. En esta ejecución, el grado 6 es un buen ejemplo.
3. **Modelo demasiado complejo:** grados elevados pueden alcanzar un R^2 de entrenamiento cercano a 1 mientras su R^2 de prueba cae considerablemente, indicando *overfitting*.

La principal lección es que **un mayor grado polinomial no implica automáticamente un mejor modelo**. La selección del grado debe considerar la capacidad de generalización, no solamente qué tan bien se ajusta el modelo a los datos de entrenamiento.

Además, la regularización mediante Lasso ofrece una herramienta adicional para controlar modelos polinomiales de alta complejidad.